employees to company it would be called a Many to One relationship, as many employees can have one company.

The way this would work with a database is that each company would be given a unique ID, called its primary key. Often this is just a serial number that increases with the addition of each company. Employees likewise are given a unique primary key ID. In the table were we store data about individual employees we also have a column that references this company ID.

So if you encounter a company with a unique ID for 17, and want a list of all its employees, you would simply ask the database to fetch a list of all employees that have a company ID equal to 17. Likewise if you look up an employee and want to know the name of the company they work for, just ask the database to pull details about a company with a unique ID that is equal to the ID stored in this employee's company column.

The way we can implement this in Django is through the ForeignKey field. So having a ForeignKey to a company in an employee model is us telling Django that there is a Many-to-One relationship between employees and companies. This is what we have also done with articles and users in our article model.

There are other types of relationships that data can have though. For instance we can have a Many to Many relationship as well. Consider the relationship between customers and companies. A company has multiple customers — unless there is a monopsony — and a customer can be a patron of multiple companies — unless there is a monopoly.

The way this would be implemented in a relational database, is through the use of a third table that defines the relationship between companies and customers. This table would only need to have two columns, a company ID and a customer ID. Then we can create a link between a company and a customer by having a row in this table that has their IDs in it.

Now if we want to look up all the customers of a company with an ID of 29, we tell the database, look up the IDs of all the customers that have an entry in the customer-company link table where their company ID is 29; and for each of these IDs look up the customer's details in the customer table.

We could as easily go the other way. For a customer with ID 1973, look up all the companies IDs that are linked to the customer ID 1973 in the customer-company link table, and for each of these companies fetch the details from the company table.

In Django this is modelled using a ManyToManyField. Tags for instance can be associated with multiple articles, and an article can have multiple tags. So we need to link an Article with its tags using a ManyToManyField in the